

Git: Push & Branching Practice Lab — Questions (Set 2)

Q1. Simple Interest Calculator Program

Write a Python file `interest.py` with one function `simple_interest(principal, rate, time)` that returns $(\text{principal} * \text{rate} * \text{time}) / 100$, plus a `__main__` block that tests it with `principal=1000`, `rate=5`, `time=2`. Run the file once to confirm the output.

- Initialize a Git repository, check the file status, stage `interest.py`, commit it with a meaningful message, rename the branch to `main`, connect it to a GitHub repository named `interest-calc` as origin, and push with the upstream set.
- Create a branch `feature-compound-interest`, add a function `compound_interest(principal, rate, time)` to the file, and commit the change on that branch.
- Create a branch `feature-total-amount`, add a function `total_amount(principal, interest)` that returns `principal + interest`, and commit the change on that branch.
- Switch back to `main`, merge both feature branches into it one after another, and push the updated `main` to GitHub.

Q2. Employee Salary Calculator Program

Write a Python file `salary.py` with a function `basic_pay(hours, rate)` that returns `hours * rate`, and a `__main__` block that tests it with `hours=160`, `rate=250`. Run it once to confirm the output.

- Initialize the repository, run `git status` before and after staging the file, commit it, set the branch to `main`, add a remote origin pointing to a GitHub repository named `salary-app`, and push with the upstream set.
- Create branch `feature-hra`, add a function `add_hra(basic, percent)` that adds house rent allowance to the basic pay, and commit it on that branch.
- Create branch `feature-deduction`, add a function `deduct_tax(salary, percent)` that subtracts a tax percentage from the salary, and commit it on that branch.
- List all branches along with their last commit using `git branch -v`, then merge both feature branches into `main` in order, and push the final `main` to GitHub.

Q3. Area Calculator Program

Write a Python file `area.py` with a function `rectangle_area(length, breadth)` that returns `length * breadth`, and a `__main__` block that tests it with `length=10`, `breadth=5`. Run it once to confirm the output.

- Set your Git identity (name and email), initialize the repository, stage and commit the file, rename the branch to `main`, add origin for a GitHub repository named `area-calculator`, and push.
- Create branch `feature-circle-area`, add a function `circle_area(radius)` that returns $3.14 * \text{radius} * \text{radius}$, and commit it on that branch.
- Create branch `feature-triangle-area`, add a function `triangle_area(base, height)` that returns $0.5 * \text{base} * \text{height}$, and commit it on that branch.
- View the commit history with `git log --oneline` before merging, then merge both feature branches into `main` and push the result.

Q4. Password Strength Checker Program

Write a Python file `password.py` with a function `has_min_length(password)` that returns `True` if the password has 8 or more characters, and a `__main__` block that tests it on the string `"vitellore"`. Run it once to confirm the output.

- Initialize the repository, stage and commit the file, rename the branch to `main`, connect it to a GitHub repository named `password-checker` as origin, and push with the upstream set.

- b) Create branch feature-has-digit, add a function has_digit(password) that returns True if the password contains at least one number, and commit it on that branch.
- c) Create branch feature-has-special-char, add a function has_special_char(password) that returns True if the password contains a special character such as @ or #, and commit it on that branch.
- d) Switch to main, view a graph of both branches with git log --oneline --graph --all, merge both feature branches into main, and push the result.

Q5. Movie Ticket Booking Program

Write a Python file booking.py with a function book_seats(available, requested) that returns the remaining seats after booking, and a __main__ block that starts with 50 available seats and books 4. Run it once to confirm the output.

- a) Initialize the repository, stage and commit the file, rename the branch to main, add a remote for a GitHub repository named ticket-booking, and push.
- b) Create branch feature-cancel-booking, add a function cancel_seats(available, cancelled) that adds cancelled seats back to the available count, and commit it on that branch.
- c) Create branch feature-ticket-price, add a function total_price(seats, price_per_seat) that returns the total cost, and commit it on that branch.
- d) Merge feature-cancel-booking into main first, then merge feature-ticket-price. If both branches had changed the same line and the second merge produced a conflict, what would you do in VS Code before completing the commit? Push the final result once resolved.

Q6. Weather Data Logger Program

Write a Python file weather.py with a function average_temp(readings) that takes a list of temperature readings and returns their average, plus a __main__ block that tests it on [30, 32, 29, 31]. Run it once to confirm the output.

- a) Initialize the repository, stage and commit the file, rename the branch to main, connect it to a GitHub repository named weather-logger as origin, and push with the upstream set.
- b) Create branch feature-max-temp, add a function max_temp(readings) that returns the highest reading, and commit it on that branch.
- c) Create branch feature-min-temp, add a function min_temp(readings) that returns the lowest reading, and commit it on that branch.
- d) Switch to main, merge both branches into main one after another, and push the final result to GitHub.
- e) Once both features are safely merged, delete both feature branches locally with git branch -d, and delete feature-min-temp from GitHub as well. What command would you use instead of -d if Git refused to delete a branch, and why does Git block the normal delete in the first place?

Q7. Contact Book Program

Write a Python file contacts.py with a function add_contact(contacts, name, phone) that adds a name-phone pair to a dictionary and returns it, plus a __main__ block that adds one contact, "Ravi" with number "9876543210", to an empty dictionary and prints it. Run it once to confirm the output.

- a) Initialize the repository, stage and commit the file, rename the branch to main, connect it to a GitHub repository named contact-book as origin, and push with the upstream set.
- b) Create branch feature-search-contact, add a function search_contact(contacts, name) that returns the phone number for a given name, and commit it on that branch.
- c) Create branch feature-delete-contact, add a function delete_contact(contacts, name) that removes a contact by name, and commit it on that branch.

d) Switch to main, merge both branches into main one after another, and push the final result to GitHub.